

# Chapitre 1

## Introduction à la Compilation

### Théorie des Langages & Compilation

Dr Tegawendé F. BISSYANDÉ

Master Informatique – UFR/SEA  
Université Joseph Ki-Zerbo



# Plan du chapitre

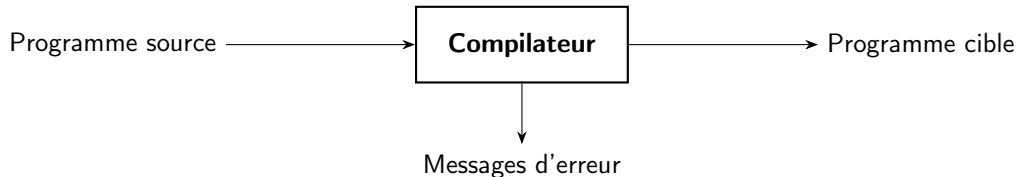
- 1 Définition d'un compilateur
- 2 Étapes de la compilation
- 3 Phases d'analyse
- 4 Synthèse du code cible
- 5 Organisation d'un compilateur
- 6 Table des symboles et traitement des erreurs
- 7 Interprètes et autres outils
- 8 Compléments
- 9 Exercices

# Qu'est-ce qu'un compilateur ?

## Définition

Un **compilateur** est un programme qui *lit* un programme écrit dans un premier langage (le **langage source**) et le *traduit* en un programme équivalent dans un autre langage (le **langage cible**).

Le compilateur doit aussi vérifier que le programme a un certain sens et **signaler les erreurs** qu'il détecte.



# Exemples de compilateurs célèbres

- **GCC** (GNU Compiler Collection) : C, C++, Fortran, ...
- **Clang/LLVM** : C, C++, Objective-C
- **javac** : Java  $\rightarrow$  bytecode JVM
- **ghc** : Haskell
- **ocamlopt** : OCaml  $\rightarrow$  code natif
- **rustc** : Rust

Compilateurs *source-à-source* (transpileurs) :

- TypeScript  $\rightarrow$  JavaScript
- CoffeeScript  $\rightarrow$  JavaScript
- Cython  $\rightarrow$  C

Il y a deux grandes parties dans la compilation :

## La partie **analyse** (front-end)

Partitionne le programme source en ses constituants et en crée une **représentation intermédiaire**.

## La partie **synthèse** (back-end)

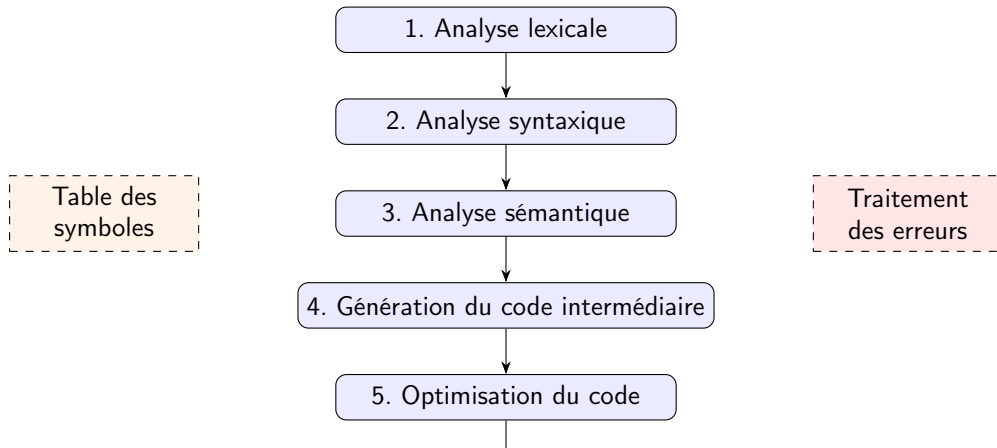
Construit le programme cible à partir de cette représentation intermédiaire.

# Les six phases d'un compilateur

Conceptuellement un compilateur opère en **6 phases**.

Chacune transforme le programme source d'une représentation en une autre.

En outre, **2 composants transversaux** interagissent avec ces 6 phases.



## Principe

Le flot de caractères formant le programme source est lu de gauche à droite et **groupé** en *lexèmes* (mots), qui sont des suites de caractères ayant une signification collective.

**Exemple** – Soit l'instruction :  $A = B1 + B2 * 60$

- 1 L'identificateur "A"
- 2 Le symbole d'affectation "="
- 3 L'identificateur "B1"
- 4 Le signe "+"
- 5 L'identificateur "B2"
- 6 Le signe de multiplication "\*"
- 7 Le nombre "60"

*Remarque* : les blancs qui séparent les mots sont éliminés.

## Principe

Les unités lexicales sont regroupées hiérarchiquement dans des collections imbriquées (phrases) ayant une signification collective, formant un **arbre syntaxique**.

Les structures grammaticales sont généralement exprimées par des **règles récursives**.

### Règles de grammaire pour les expressions :

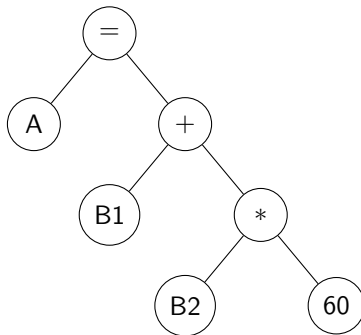
- Règles de base : *Tout identificateur est une expression ; Tout nombre est une expression.*
- Règle récursive : Si  $e_1$  et  $e_2$  sont des expressions, alors  $e_1+e_2$ ,  $e_1*e_2$  et  $(e_1)$  sont aussi des expressions.



# Arbre abstrait (Abstract Syntax Tree)

Pendant l'analyse, les opérations spécifiées par le programme source sont déterminées et conservées dans une structure hiérarchique appelée **arbre abstrait**.

Un arbre abstrait est constitué de **nœuds** qui représentent les *opérations* et les **filles des nœuds** qui représentent les *arguments* des opérations.



Arbre abstrait pour  $A = B1 + B2 * 60$

## Principe

Contrôle pour s'assurer que l'assemblage des constituants du programme **a un sens**.

Un constituant important est le **contrôle de type**.

**Exemple :** Si nous déclarons A, B1 et B2 des réels, alors nous devons convertir l'entier 60 en réel 60.0.

Cette conversion est représentée dans l'arbre par un nœud `entier_vers_réel`.

## Code à trois adresses

On génère un code intermédiaire où chaque instruction a **au plus trois opérandes**.

**Exemple** pour  $A = B1 + B2 * 60$  (avec B2 réel) :

```
temp1 := entier_vers_reel(60);  
temp2 := id3 * temp1;  
temp3 := id2 + temp2;  
id1    := temp3;
```

## Objectif

Améliorer le code intermédiaire pour que le code final s'exécute plus rapidement et utilise le **minimum de mémoire**.

## Exemple optimisé :

```
temp1 := id3 * 60.0;  
id1    := id2 + temp1;
```

Techniques classiques d'optimisation :

- Élimination des sous-expressions communes
- Propagation des constantes
- Élimination du code mort
- Déroulage de boucles

## Principe

Production du code cible en **langage d'assemblage**. L'aspect crucial est l'assignation des variables aux **registres** du processeur.

**Exemple** avec 2 registres R1 et R2 :

```
MOVF  R2, id3      ; F: virgule flottante
MULF  R2, #60.0    ; #: constante imm\'ediate
MOVF  R1, id2
ADDF  R1, R2
MOVF  id1, R1
```

# Front-end et back-end

On regroupe les phases en **deux parties** :

## Front-end (partie frontale)

Phases dépendant du **langage source** :

- Analyse lexicale
- Analyse syntaxique
- Analyse sémantique
- Table des symboles
- Code intermédiaire
- Traitement des erreurs

## Back-end (partie finale)

Phases dépendant de la **machine cible** :

- Optimisation du code intermédiaire
- Production du code final
- Gestion de la table des symboles
- Traitement des erreurs

Cette séparation permet de **réutiliser** le front-end pour différentes machines cibles et le back-end pour différents langages sources.

# Outils utilisés à chaque étape

| Phase              | Outil théorique                         | Outil logiciel          |
|--------------------|-----------------------------------------|-------------------------|
| Analyse lexicale   | Expressions régulières, automates finis | Lex, Flex, OCamllex     |
| Analyse syntaxique | Grammaires, automates à pile            | Yacc, Bison, OCamllyacc |
| Analyse sémantique | Règles de typage                        | –                       |
| Code intermédiaire | Traduction dirigée par la syntaxe       | –                       |
| Optimisation       | Analyse de flots de données             | –                       |
| Génération de code | Allocation de registres                 | –                       |

## Définition

Une table des symboles est une structure de données contenant un enregistrement pour chaque **identificateur**, muni de champs pour ses attributs : *emplacement mémoire*, *type*, *portée*, etc.

Pour les fonctions, la table des symboles garde :

- Le **nombre** et les **types** de ses arguments
- Le **mode de passage** de chacun d'eux (valeur, référence)
- La **valeur de retour**

La table des symboles est consultée et mise à jour à **chaque phase** de la compilation.



Chaque phase peut rencontrer des erreurs :

- **Erreurs lexicales** : caractère illégal, chaîne non terminée
- **Erreurs syntaxiques** : parenthèse manquante, point-virgule oublié
- **Erreurs sémantiques** : types incompatibles, variable non déclarée
- **Erreurs de génération** : dépassement de capacité

## Principe de récupération

Après avoir détecté une erreur, une phase doit la traiter de telle façon que la compilation puisse **continuer** et que d'autres erreurs puissent être détectées.

# Compilateur vs. interprète

## Compilateur

- Traduit le programme source en programme cible
- Le programme cible est exécuté ultérieurement
- Exécution rapide
- Exemples : GCC, javac, rustc

## Interprète

- Exécute directement les opérations spécifiées par le programme source
- Pas de programme cible généré
- Plus flexible, plus lent
- Exemples : Python, Ruby, shell Unix

## Approches hybrides

Certains langages utilisent une combinaison : compilation vers un **bytecode** puis interprétation par une **machine virtuelle** (Java, Python, C#).

## Rôle

Charger les fichiers et les fonctions des bibliothèques et les **relier** pour la production du code exécutable.

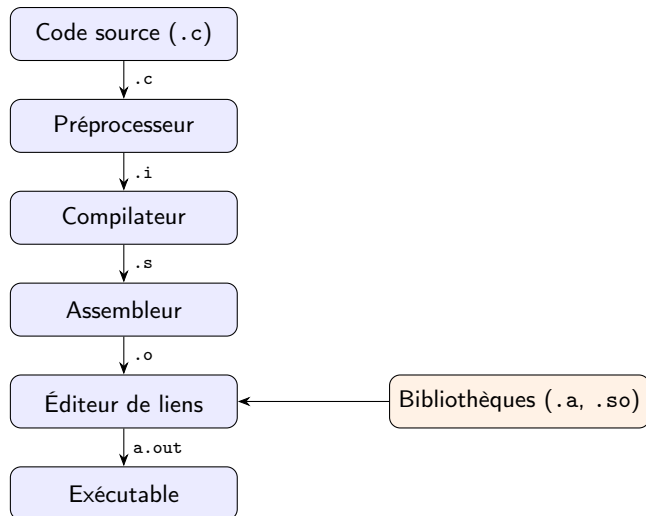
- **Édition de liens statique** : les bibliothèques sont copiées dans l'exécutable
- **Édition de liens dynamique** : les bibliothèques sont chargées à l'exécution (fichiers .so sous Linux, .dll sous Windows)

# Historique de la compilation

- **1952** : premier compilateur (A-0, Grace Hopper) – traduction de codes mathématiques en code machine
- **1957** : **FORTRAN** (John Backus, IBM) – premier compilateur optimisant, 18 personnes-années de développement
- **1960** : ALGOL 60 et la notation **BNF** (Backus-Naur Form) pour décrire formellement la syntaxe
- **1962** : premiers travaux sur l'analyse syntaxique LR (Knuth)
- **1975** : Yacc (Yet Another Compiler Compiler) – générateur d'analyseurs syntaxiques (Johnson, Bell Labs)
- **2003** : LLVM – infrastructure modulaire de compilation, base de Clang, Swift, Rust

La compilation est un domaine fondateur de l'informatique qui reste très actif (JIT, WebAssembly, compilation pour GPU, ...).

# La chaîne complète de production d'un exécutable



# Compilation croisée et compilation JIT

## Compilation croisée (cross-compilation)

Le compilateur s'exécute sur une machine *A* mais produit du code pour une machine *B* différente. Très utilisé pour les systèmes embarqués (ARM, RISC-V) et le développement mobile.

## Compilation Just-In-Time (JIT)

Le code est compilé **à l'exécution**, au moment où il est nécessaire. Permet des optimisations basées sur le **profil d'exécution** réel.

- **JVM HotSpot** : compile le bytecode Java en code natif pour les méthodes “chaudes”
- **V8** : moteur JavaScript de Chrome, compile en code natif
- **PyPy** : interprète Python avec JIT intégré

Le choix de la représentation intermédiaire (IR) est crucial :

## IR de haut niveau

- Arbre de syntaxe abstraite (AST)
- Proche du langage source
- Facilite les analyses sémantiques

## IR de bas niveau

- Code à trois adresses
- SSA (Static Single Assignment)
- Proche du code machine
- Facilite les optimisations

**LLVM IR** : représentation intermédiaire en forme SSA, indépendante de la machine, utilisée par de nombreux compilateurs modernes (Clang, Rust, Swift, Julia, ...).

# Exercice 1 – Questions de cours

- 1 Donnez la définition d'un compilateur. En quoi diffère-t-il d'un interprète ?
- 2 Citez les six phases d'un compilateur et décrivez brièvement le rôle de chacune.
- 3 Quelle est la différence entre le front-end et le back-end d'un compilateur ? Quel est l'intérêt de cette séparation ?
- 4 Quel est le rôle de la table des symboles ? À quels moments est-elle utilisée ?



## Exercice 2 – Analyse d'une instruction

Soit l'instruction suivante :

$$\text{result} = (x + 3) * y - z / 2$$

- ➊ Effectuez l'**analyse lexicale** : identifiez tous les lexèmes et leur catégorie (identificateur, opérateur, nombre, séparateur).
- ➋ Dessinez l'**arbre syntaxique** correspondant, en respectant les priorités des opérateurs.
- ➌ Dessinez l'**arbre abstrait** correspondant.
- ➍ Si  $x$ ,  $y$ ,  $z$  sont des `float` et que 3 et 2 sont des entiers, quelles conversions l'analyse sémantique doit-elle insérer ?

## Exercice 3 – Génération de code intermédiaire

Soit l'expression :  $a = b * c + b * c$

- 1 Générez le code intermédiaire à trois adresses naïf.
- 2 Proposez une optimisation en éliminant la sous-expression commune.
- 3 Traduisez le code optimisé en instructions assembleur simplifiées (avec des registres R1, R2, R3).

**Exercice 4** – Classez les erreurs suivantes selon la phase de compilation qui les détecte :

- 1 Utilisation d'un caractère # non autorisé dans un identificateur
- 2 Parenthèse fermante manquante dans une expression
- 3 Addition d'un entier et d'une chaîne de caractères
- 4 Variable utilisée mais non déclarée